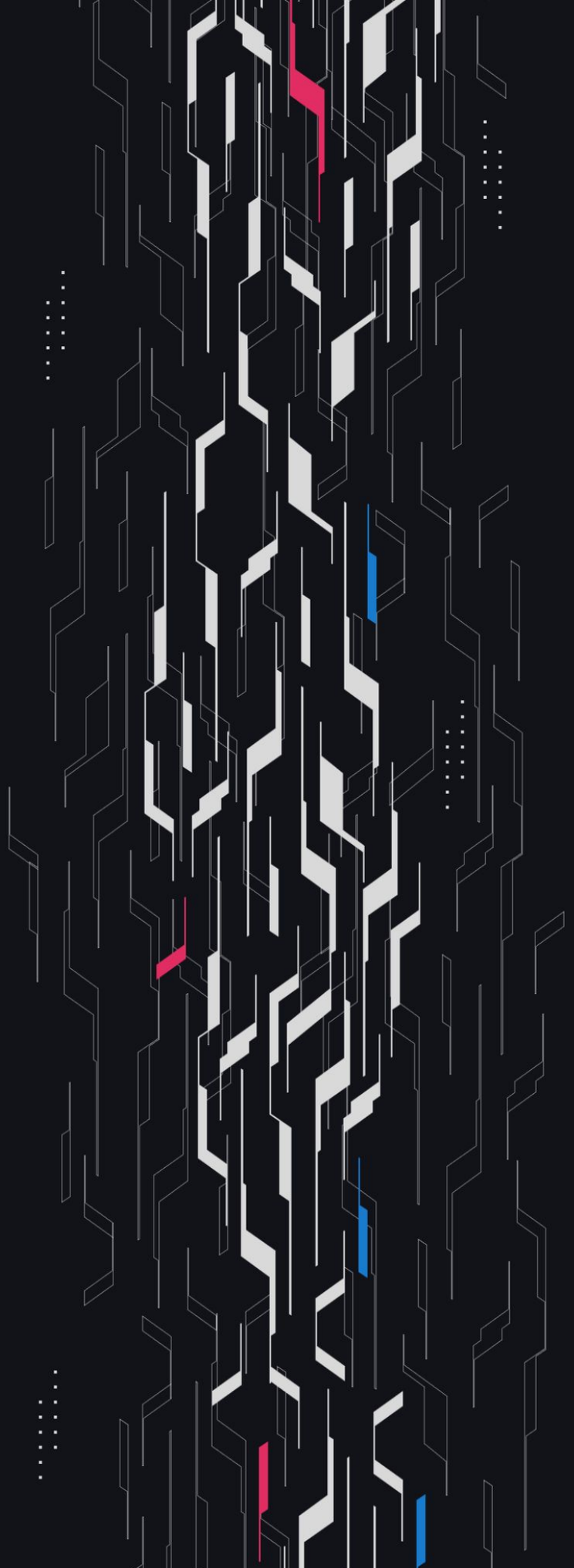
GA GUARDIAN

Ethena

Onchain Minter

Security Assessment

December 2nd, 2025



Summary

Audit Firm Guardian

Prepared By Owen Thurm, Zdravko Hristov, Michael Lett, Bounty MO

Client Firm Ethena

Final Report Date December 2, 2025

Audit Summary

Ethena engaged Guardian to review the security of their Ethena's Onchain Minter. From the 6th of October to the 15th of October, a team of 4 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the lack of critical issues detected and minimal code changes following the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only Low or Medium-severity issues were discovered. Design patterns are sound, and test coverage is reasonable. Small changes, such as modifying rounding logic, may introduce new vulnerabilities and should be carefully reviewed. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0 High/Critical findings. Varied Low/Medium severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1 High finding and ≥ 3 Medium. Varied Low severity findings.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities (~2/week). Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Overview

Project Summary

Project Name	Ethena
Language	Solidity
Codebase	https://github.com/ethena-labs/onchain-minting-internal
Commit(s)	Main Review commit: fcfd59c7b75944bc2751e0cdaa3aafe2996dfc99 Remediation Review commit: 05aff055028b62f171bee39e67dd23bfde86516c

Audit Summary

Delivery Date	December 2, 2025
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	0	0	0	0	0	0
Medium	1	0	0	0	0	1
Low	19	0	0	8	0	11
Info	52	0	0	17	0	35

Audit Scope & Methodology

```
contract,source,total,comment
src/token/IxUSD.sol,13,69,44
src/token/xUSD.sol,74,221,125
src/oracle/AggregateOracleFeed.sol,126,260,99
src/oracle/ChainlinkOracleFeed.sol,28,99,55
src/oracle/IAggregateOracleFeed.sol,22,148,103
src/oracle/IOracleFeed.sol,6,49,39
src/oracle/PythOracleFeed.sol,47,141,71
src/oft/OFTErrors.sol,16,86,56
src/oft/OFTEvents.sol,17,110,77
src/oft/OFTOwnable2StepUpgradeable.sol,47,124,66
src/oft/OFTRateLimiterUpgradeable.sol,66,157,74
src/oft/xUSDOFTUpgradeable.sol,155,398,209
src/minting/IERC20Mintable.sol,5,23,16
src/minting/IOnChainMinting.sol,199,814,514
src/minting/OnChainMinting.sol,876,1701,658
src/access/ISingleAdminAccessControl.sol,7,38,27
src/access/SingleAdminAccessControl.sol,45,165,102
source count: {
  total: 4603,
  source: 1749,
  comment: 2335,
  single: 126,
  block: 2209,
  mixed: 0,
  empty: 519,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 1.3350485991995427
}
```

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	🔴 Critical	🟠 High	🟡 Medium
Likelihood: *Medium*	🟠 High	🟡 Medium	🟢 Low
Likelihood: *Low*	🟡 Medium	🟢 Low	🟢 Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Invariants Assessed

During Guardian's review of Ethena, fuzz-testing was performed on the protocol's main functionalities. Given the dynamic interactions and the potential for unforeseen edge cases in the protocol, fuzz-testing was imperative to verify the integrity of several system invariants.

Throughout the engagement the following invariants were assessed for a total of 10,000,000+ runs with a prepared fuzzing suite.

ID	Description	Tested	Passed	Remediation	Run Count
OFT-01	After a successful OFT send, the amount in flight should be less than or equal to the rate limit	✅	✅	✅	10m+
OFT-02	After a successful OFT send, the amount in flight should increase with the amountLD sent	✅	✅	✅	10m+
MINTING-01	After a successful mint, mintedInEpoch and mintedInPeriod should increase with the same amount for all three states (global, collateral, benefactor)	✅	✅	✅	10m+
MINTING-02	After a successful redeem, redeemedInEpoch and redeemedInPeriod should increase with the same amount for all three states (global, collateral, benefactor)	✅	✅	✅	10m+
MINTING-03	After a successful mint, mintedInEpoch and mintedInPeriod should increase with the result of getQuote (global, collateral, benefactor)	✅	✅	✅	10m+
MINTING-04	After a successful redeem, redeemedInEpoch and redeemedInPeriod should increase with the input amount (global, collateral, benefactor)	✅	✅	✅	10m+
MINTING-05	After a successful mint, the benefactor collateral balance should decrease with order.amountIn	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
MINTING-06	After a successful mint, the beneficiary xUSD balance should increase with the result of getQuote	✅	✅	✅	10m+
MINTING-07	After a successful redeem, the benefactor xUSD balance should decrease with order.amountIn	✅	✅	✅	10m+
MINTING-08	After a successful redeem, the beneficiary collateral balance should increase with the result of getQuote	✅	✅	✅	10m+
MINTING-09	The minted assets with a given collateral for an epoch or period should not exceed the globally minted assets for that epoch or period	✅	✅	✅	10m+
MINTING-10	The redeemed assets for a given collateral for an epoch or period should not exceed the globally redeemed assets for that epoch or period	✅	✅	✅	10m+
MINTING-11	The minted assets by a given benefactor for an epoch or period should not exceed the globally minted assets for that epoch or period	✅	✅	✅	10m+
MINTING-12	The redeemed assets by a given benefactor for an epoch or period should not exceed the globally redeemed assets for that epoch or period	✅	✅	✅	10m+
MINTING-13	After executing an order, the current epoch and period should be the same for each state (global, collateral, benefactor)	✅	✅	✅	10m+
MINTING-14	After a successful mint, if the epoch/period was rolled, mintedInEpoch/mintedInPeriod should be the result of getQuote for all three states (global, collateral, benefactor)	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
MINTING-15	After a successful redeem, if the epoch/period was rolled, redeemedInEpoch/redeemedInPeriod should be the input amount for all three states (global, collateral, benefactor)	✅	✅	✅	10m+

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Future Prices Allowed For Chainlink	Validation	Low	Resolved
L-02	burnFrom Discrepancy In xUSD Implementations	Compatibility	Low	Resolved
L-03	Misleading RateLimiterUpgradeableStorageLocation	Unexpected Behavior	Low	Resolved
L-04	Rate Limit Consumed With No Input	Unexpected Behavior	Low	Acknowledged
L-05	Incorrect RL State Preservation During Updates	Unexpected Behavior	Low	Acknowledged
L-06	Timestamp Preservation Is In Favor Of The Users	Unexpected Behavior	Low	Acknowledged
L-07	transferToCustody() Doesn't Check The Recipient	Validation	Low	Resolved
L-08	Period Limits Can Be Set Lower Than Epoch Limits	DoS	Low	Acknowledged
L-09	Oracle Bounds May Bias The Oracle	Unexpected Behavior	Low	Resolved
L-10	Oracle Specific Staleness Thresholds Hardcoded	Validation	Low	Resolved
L-11	Aggregator Updates Are Sandwichable	Gaming	Low	Acknowledged
L-12	Limits Ineffectively Prevent Concentration	Validation	Low	Acknowledged
L-13	Shared Nonce Breaks Integrations	Compatibility	Low	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
L-14	Redstone Max Staleness Is 30 Hours	Unexpected Behavior	Low	Resolved
L-15	Peg Arb Affected By Fees	Unexpected Behavior	Low	Acknowledged
L-16	Fee Is Applied Inconsistently	Unexpected Behavior	Low	Acknowledged
L-17	getBenefactorFeesForCollateral Skips Defaults	Unexpected Behavior	Low	Resolved
I-01	Oracle Manipulation With Inappropriate Staleness	Gaming	Info	Resolved
I-02	Ethena Minter Used To Exit Depegged Collateral	Warning	Info	Acknowledged
I-03	Unnecessary Zero Configuration Handling	Gas Optimization	Info	Resolved
I-04	Duplicated Functions	Unexpected Behavior	Info	Resolved
I-05	defaultBenefactorMaxRedeemPerPeriod Typo	Typo	Info	Resolved
I-06	DEFAULT_ADMIN_ROLE Restriction Bypass	Access Control	Info	Acknowledged
I-07	OFT's Minters() Function Returns Minter Status	Best Practices	Info	Resolved
I-08	Inconsistency Between Tokens Mint Functions	Best Practices	Info	Resolved
I-09	Burned Events Emitted Only For burnFrom()	Events	Info	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
I-10	0xdead Shouldn't Be Blacklisted	Informational	Info	Acknowledged
I-11	AggregateOracleFeed Always Rounds Down	Best Practices	Info	Acknowledged
I-12	USDT Has A Fee On Transfer Feature	Unexpected Behavior	Info	Acknowledged
I-13	Lack Of Admin Validation In PythOracleFeed	Validation	Info	Resolved
I-14	A Single Oracle Can Cause OOG Revert	Informational	Info	Acknowledged
I-15	OracleStale Emitted For Prices In Future	Best Practices	Info	Resolved
I-16	Inaccurate NatSpec For OFT Events And Errors	Best Practices	Info	Resolved
I-17	Benefactor Mint Fee Cannot Be Changed	Validation	Info	Resolved
I-18	Event Parameters Documentation Mismatch	Best Practices	Info	Resolved
I-19	Fee Features Unsupported	Documentation	Info	Resolved
I-20	Lacking SafeCast Usage	Best Practices	Info	Resolved
I-21	addCollateral Can Be Used Inappropriately	Unexpected Behavior	Info	Resolved
I-22	removeCollateral Unexpectedly No-ops	Unexpected Behavior	Info	Acknowledged

Findings & Resolutions

ID	Title	Category	Severity	Status
I-23	Missing Benefactor Address Validation	Validation	Info	Acknowledged
I-24	Opaque BenefactorConfigUpdated Event	Events	Info	Acknowledged
I-25	Missing Signer Address Validation	Validation	Info	Acknowledged
I-26	rescueRecipient Can Be Blacklisted	Informational	Info	Resolved
I-27	Total Minted/redeemed Per State Going Over Limit	Informational	Info	Acknowledged
I-28	Limit Reverts May Be Duplicated	Best Practices	Info	Resolved
I-29	Fee Collection Rounds Down	Rounding	Info	Resolved
I-30	Benefactor Addresses Must Handle Tokens	Validation	Info	Resolved
I-31	GetQuote() Does Not Validate Price Freshness	Math	Info	Resolved
I-32	Collateral Decimals Can Be Updated	Validation	Info	Resolved
I-33	getQuote Can Be Marked External	Best Practices	Info	Resolved
I-34	Unnecessary Optimizations	Informational	Info	Acknowledged
I-35	End Timestamp Functions Return The Next Start	Informational	Info	Acknowledged

Findings & Resolutions

ID	Title	Category	Severity	Status
I-36	COLLATERAL_MANAGER_ROLE Can Change The Oracle	Trust Assumptions	Info	Acknowledged
I-37	Beneficiary Can Be Set For Inactive Benefactor	Informational	Info	Acknowledged
I-38	Regular xUSD Does Not Implement Blacklist	Warning	Info	Resolved
I-39	Fee Exempt Mints Reduce Collateralization	Warning	Info	Acknowledged
I-40	Peg Updates Are Sandwichable	Gaming	Info	Acknowledged

L-01 | Future Prices Allowed For Chainlink

Category	Severity	Location	Status
Validation	Low	ChainlinkOracleFeed.sol	Resolved

Description

In the `ChainlinkAggregatorFeed` contract the same future price handling exists as in the `PythOracleFeed` contract.

However because Chainlink is an aggregator oracle where the updated at time is assigned based on the `block.timestamp` of the update transaction.

Therefore, it is not possible for a future dated price timestamp to be reported.

Recommendation

Consider removing the future price timestamp handling and require that a price is strictly from the current timestamp or in the past up to the staleness threshold.

Resolution

Ethena Team: Resolved.

L-02 | burnFrom Discrepancy In xUSD Implementations

Category	Severity	Location	Status
Compatibility	Low	Global	Resolved

Description

In the non-OFT xUSD implementation the `burnFrom` function is permissioned such that only callers who are whitelisted minters may invoke it.

However the `ERC20BurnableUpgradeable.burnFrom` function includes an allowance validation mechanism, and for this reason the same `burnFrom` function in the `xUSDOFTUpgradeable` contract is not permissioned to only minter addresses.

Recommendation

Consider if both the normal xUSD and `xUSDOFTUpgradeable` implementations should standardize on the same permissions for the `burnFrom` function, whether that be only for minters or callable for the public.

If the intent is that only minters may call the `burnFrom` function, then consider if the approval validation is necessary and whether minters should have the ability to burn without being limited by the approval amount, as they are also trusted to mint a nearly unlimited amount of the token.

Resolution

Ethena Team: Resolved.

L-03 | Misleading RateLimiterUpgradeableStorageLocation

Category	Severity	Location	Status
Unexpected Behavior	Low	RateLimiterUpgradeable.sol	Resolved

Description

In the `RateLimiterUpgradeable` contract the storage location for the `RateLimiterUpgradeableStorageLocation` value does not agree with what the comment proposes derives it.

The proposed `keccak256(abi.encode(uint256(keccak256("RateLimiterUpgradeable")) - 1)) & ~bytes32(uint256(0xff))` result is:

`0xf40fb8ff74ec76132acbfc7d3a15179466240c00e91ec9e6b83495e3a3e51600`.

However the actual storage location value used is:

`0x8327c26dd5e5ceb4f0d1337dcb7f7536980a6eda04f5f7fb6577c3c861cded00`.

Recommendation

Consider using the actual result of the proposed:

`keccak256(abi.encode(uint256(keccak256("RateLimiterUpgradeable")) - 1)) & ~bytes32(uint256(0xff))` value,

or update the comment to indicate what the current value represents.

Resolution

Ethena Team: Resolved.

L-04 | Rate Limit Consumed With No Input

Category	Severity	Location	Status
Unexpected Behavior	Low	xUSDOFTUpgradeable.sol: 383	Acknowledged

Description PoC

`xUSDOFTUpgradeable` inherits from the `RateLimiterUpgradeable` contract and implements a rate limiting - for a given period of time users can send only limited amount of tokens to other chains.

The `_debit()` function is run on each crosschain call to reduce the balance of the sender. The function is overridden to check the rate limit and update the amount that's sent.

Notice that the call to `_checkAndUpdateRateLimit()` passes `_amountLD` as amount. Inside that function, `amountInFlight` will be increased by the value of this parameter.

After the rate limit logic is executed, only the amount returned by `_debitView()` will be burned from the user.

The `debitView()` will return amount equivalent to the return value of `_removeDust()`.

Since `localDecimals = 18` and `sharedDecimals = 6`, `decimalConversionRate = 12`. Therefore, the last 12 digits of the `amountLD` will be cut off.

If a user passes `decimalConversionRate - 1` as `amountLD`, the burned amount will be `0`, but the actual `amountLD` will be used from the available limit. This can be performed even by users that don't have any tokens.

However, to use up the whole limit is not that easy because of gas cost. Still, for smaller limits or almost used ones, this attack may prevent crosschain communication when not expected.

Recommendation

Use the actual `amountSentLD` when updating the rate limit parameters.

Resolution

Ethena Team: Acknowledged.

L-05 | Incorrect RL State Preservation During Updates

Category	Severity	Location	Status
Unexpected Behavior	● Low	RateLimiterUpgradeable.sol	Acknowledged

Description PoC

RateLimiterUpgradeable stores information about each eid in the following struct:

```
struct RateLimit {
uint256 amountInFlight;
uint256 lastUpdated;
uint256 limit;
uint256 window;
}
```

The limit/window represents the rate of tokens that can be sent to the destination chain per second. The system uses amountInFlight to track the debited assets over the window. As time passes, the amountInFlight experiences decay which allows more tokens to be sent.

As described in Rate Limit State Preservation During Updates, when _setRateLimits is invoked to change the limits, the in flight amount is not reset to 0 because if it was, users would have been able to immediately consume the new limit, even if the old one was already used up.

This behavior creates a new issue that results in users not being able to send tokens for more than the configured window. Let's look at the example in the README.

Like step 3 says, user locked out until decay reduces it below 10. However, the decay rate is now 10/hour, which means 90 hours must pass before the in flight amount reaches 0. It may be expected that the if branch in _amountCanBeSent() will reset the in flight amount to 0 once the window is over, but nothing stops users from sending 0 amount requests to reset the _lastUpdated value and never enter the conditional's body.

In result, after lowering the limit, users have to wait a longer, unexpected period of time than the expected configured window.

This also doesn't align with what's stated in the README
> Natural Decay: The time-based decay will naturally resolve the situation over the configured window

Recommendation

When updating the limit to a lower value than the current one via a call to _setRateLimits(), consider:
• setting the current amountInFlight to min(amountInFlight, newLimit)
• setting lastUpdated to block.timestamp

This approach will ensure users are subject to the new limit rate and at the same time are unable to immediately start spending it.

NOTE: If this fix is implemented, the owner will be able to pass limit = 0, to reset the in flight value.

Resolution

Ethena Team: Acknowledged.

L-06 | Timestamp Preservation Is In Favor Of The Users

Category	Severity	Location	Status
Unexpected Behavior	Low	RateLimiterUpgradeable.sol	Acknowledged

Description

It's stated in the README that the _setRateLimit() "function intentionally does NOT reset amountInFlight or lastUpdated values for existing rate limits". It's said that the approach "reflects conservative rate limiting principles".

But if we take a look at the decay, we can see that the larger the timeSinceLastDeposit, the larger the decay itself.

```
uint256 decay = (_limit * timeSinceLastDeposit) / _window;
```

The bigger the decay is, the more assets can be sent to the destination chain, which is the opposite of the expected conservative behavior defined in the README. This also grants users the ability to send more tokens than the previous limit if the new rate is larger.

For example:

• RL = 10 000 tokens / 10 hours
• User A sends 10 000 tokens
• 2 hours pass without activity
• New RL is set to 5000 tokens / 2 hours
• Users are now able to immediately send 5000 tokens

Recommendation

Consider resetting the timestamp on RL modification, or at least when the new limit / window is greater than the previous ratio.

Resolution

Ethena Team: Acknowledged.

L-07 | transferToCustody() Doesn't Check The Recipient

Category	Severity	Location	Status
Validation	Low	OnChainMinting.sol: 347	Resolved

Description

`OnChainMinting.transferToCustody()` allows the `COLLATERAL_MANAGER_ROLE` to send any collateral tokens from the minting contract to the collateral custodian.

The problem with this function is that it doesn't check if `custodianAddress` is `address(0)`. This can lead to loss of funds if the function is misused.

For example, if `removeCollateral()` is executed before that, the collateral config, including the custodian, will be deleted and the funds will be just sent to `address(0)`.

Recommendation

Revert in `transferToCustody()` if the recipient is `address(0)`.

Resolution

Ethena Team: Resolved.

L-08 | Period Limits Can Be Set Lower Than Epoch Limits

Category	Severity	Location	Status
DoS	● Low	src/minting/OnChainMinting.sol` (lines 748-760, 914-926, 604-614, 625-635, 1004-1014, 1025-1035)	Acknowledged

Description

The protocol has no validation ensuring that period limits are greater than or equal to epoch limits.

Since an epoch is a subset of time within a period, it's logically impossible to mint/redeem more in a single epoch than in the entire period.

However, the admin can set `maxMintPerEpoch > maxMintPerPeriod`, creating an impossible constraint that breaks the system.

The same issue exists for:
• Collateral-specific limits
• Benefactor-specific limits
• Default benefactor limits

In the `README` it's mentioned that this is expected, but the code comments and max durations for epochs and periods don't align with that

```
/// @notice Minimum allowed epoch duration (10 seconds)
/// @dev Epochs are intended to track shorter time periods, such as 30 seconds, while periods track longer durations
uint256 private constant MIN_EPOCH_DURATION = 10;
/// @notice Maximum allowed epoch duration (24 hours)
uint256 private constant MAX_EPOCH_DURATION = 24 hours;
/// @notice Minimum allowed period duration (10 seconds)
/// @dev Periods are intended to track longer time periods, such as 24 hours, complementing the shorter epoch-based limits
uint256 private constant MIN_PERIOD_DURATION = 10;
/// @notice Maximum allowed period duration (30 days)
uint256 private constant MAX_PERIOD_DURATION = 30 days;
```

Recommendation

Add validation during the initial setup to avoid the period limit being less than the epoch limit.
Consider adding validation for when the admin updates the period and epoch limits through the `setGlobalPeriodLimits()` function such that these limits do not contradict each other.

And finally, consider validating that a period is longer than an epoch in the `setEpochDuration` and `setPeriodDuration` functions.

Resolution

Ethena Team: Acknowledged.

L-09 | Oracle Bounds May Bias The Oracle

Category	Severity	Location	Status
Unexpected Behavior	Low	AggregateOracleFeed.sol	Resolved

Description

In the `AggregateOracleFeed` contract if an oracle's reported price is less than the minimum or greater than the maximum then it is skipped from the end result.

In some exceptional scenarios this may lead to an unintended bias of the `AggregateOracleFeed` result and allow for a small mispricing.

Consider the following example:
• `minNumberOfOracles` is 2
• There are 4 configured oracles
• The `minOraclePrice` is 0.98 and the `maxOraclePrice` is 1.02
• The stablecoin in question deepens slightly to a market average price of $0.981
• Oracles 1 & 2 report 0.982 and 0.984 respectively
• Oracles 3 & 4 report 0.979 and 0.978 respectively
• The average considering all oracle results is (0.982 + 0.984 + 0.979 + 0.978) / 4 = 0.981
• However oracles 3 & 4 are removed from the average result since they happen to lie below the minOraclePrice
• The actual result from the `AggregateOracleFeed` is (0.982 + 0.984) / 2 = 0.983
• This produces a less accurate price and biases the oracle upwards in this case

Recommendation

Consider if this bias in such edge case scenarios is acceptable. This edge case may inform the configuration of the `minOraclePrice` and `maxOraclePrice`, it should only validate against extreme values that are highly likely to be aberrations.

Otherwise consider removing the need to exclude oracles based on individual price results that are above an arbitrary min/max and instead use the median price reported by an oracle.

Resolution

Ethena Team: Resolved.

L-10 | Oracle Specific Staleness Thresholds Hardcoded

Category	Severity	Location	Status
Validation	Low	ChainlinkOracleFeed.sol	Resolved

Description

In the `ChainlinkOracleFeed` contract the staleness threshold is hardcoded to be 1 day and 1 minute long, however some feeds have a different heartbeat than this such as the ETH-BTC feed on Ethereum mainnet as an example.

This means the `ChainlinkOracleFeed` contract is incompatible with such feeds, as it will either not give enough time for an oracle update to occur if the heartbeat is longer than 24 hours, or it will give too much time for an oracle update to occur if it is less than 24 hours.

Recommendation

Consider allowing the `ORACLE_STALENESS_THRESHOLD` value to instead be assigned on deployment and perhaps even configurable afterwards.

Resolution

Ethena Team: Resolved.

L-11 | Aggregator Updates Are Sandwichable

Category	Severity	Location	Status
Gaming	Low	Global	Acknowledged

Description

When the market price moves past an aggregator's deviation tolerance an update is triggered onchain.

For aggregators with a larger deviation threshold, this could create small arbitrage opportunities when deviation triggered updates occur.

Consider the following scenario:

• The system is deployed on a network with a public mempool, e.g. Ethereum
• The reported price for USDT from an aggregator is $1.00 (assume the `AggregateOracleFeed` reports this price directly)
• The deviation threshold for the aggregator is 1%
• The market price for USDT moves to $0.99
• User A observes a deviation triggered update in the public mempool and chooses to front-run it by depositing 100 USDT into the `OnChainMinting` contract
• The USDT is still valued at the yet-to-be-updated aggregator price of $1
• (Ignore fees) User A receives `oracleAmountOut = 100e6 * 1e18 * 1e18 / (1e18 * 1e6) = 100e18 xUSD`
• The aggregator update occurs and the reported price of USDT is now $0.99
• User A immediately back runs this and withdraws their xUSD
• (Ignore fees) User A receives `oracleAmountOut = 100e18 * 1e18 * 1e6 / (0.99e18 * 1e18) = 101.01010101 USDT`

Alice increased her USDT holdings by arbitraging the aggregator update and the way that the `OnChainMinting` system relies on it.

Recommendation

The effect of this is reduced using an aggregated oracle because a single deviation update is dampened by other oracles which are unlikely to also be updated at the same time. Be aware of this arbitrage vector when choosing the feeds which are to be supported by the Aggregator oracle, ideally those with small deviation thresholds are prioritized.

Furthermore, consider the deviation thresholds of the underlying oracles when assigning the fee rate as any fee rate that matches or exceeds the deviation threshold renders this arbitrage unprofitable.

Resolution

Ethena Team: Acknowledged.

L-12 | Limits Ineffectively Prevent Concentration

Category	Severity	Location	Status
Validation	Low	OnChainMinting.sol	Acknowledged

Description

In the `OnChainMinting` system the epoch and period limits are meant to rate limit the amount of minting and redemptions that can occur in the system as well as limit the concentration of certain collateral types that can build up.

However the `_validateCollateralEpochLimits` and `_validateCollateralPeriodLimits` validations work based on the `amountOut` for mints and `order.amountIn` for burns, which are both xUSD asset amounts.

Because of the varying fee rates applied, and in the case of any depeg scenario, this ineffectively limits the amount of collateral actually deposited into the system, negating the objective of avoiding collateral concentration.

For example:

• Consider a scenario where USDC has depegged to 90% (for explicative purposes)
• The `benefactorMaxMintPerPeriod` is 100
• Assume no fees
• For a stablecoin with no fees and no depeg, only 100 tokens can be deposited
• For USDC in this case, 100/90 ~= 111.11 USDC can be deposited
• This allows for a higher concentration of USDC to be accumulated, especially when USDC repegs

Consider another case:

• Assume a 5% fee on deposits specifically for USDC (for explicative purposes)
• The `benefactorMaxMintPerPeriod` is 100
• For a stablecoin with no fees and no depeg, only 100 tokens can be deposited
• For USDC in this case, 100/95 ~= 1.053 USDC can be deposited
• This allows for a higher concentration of USDC to be accumulated

Recommendation

For collaterals, consider also implementing a collateral amount based validation per period and epoch to effectively limit the concentration of certain tokens. Otherwise, consider adding a simple max collateral amount validation in the system.

Resolution

Ethena Team: Acknowledged.

L-13 | Shared Nonce Breaks Integrations

Category	Severity	Location	Status
Compatibility	Low	OnChainMinting.sol	Resolved

Description

One benefactor can have multiple approved `delegatedSigners` associated with it, however the `orderNonceInvalidator` is tied to the benefactor's state and is thus shared amongst all of the `delegatedSigners` as well as the benefactor themselves.

This may cause issues for complex integrations which use multiple different services to submit orders through various `delegatedSigners`.

Since each service does not have its own `orderNonceInvalidator` then there may arise race conditions that cause actions to unintentionally fail, rather than simply providing the intended benefit of preventing the same order from being submitted twice.

Recommendation

Consider keying the `orderNonceInvalidator` mapping based upon the `delegatedSigner`/benefactor who is initiating the execution.

Resolution

Ethena Team: Resolved.

L-14 | Redstone Max Staleness Is 30 Hours

Category	Severity	Location	Status
Unexpected Behavior	Low	Global	Resolved

Description

In the `MultiFeedAdapterWithoutRounds` base contract which is used for `RedStone` aggregator oracles, the on-chain validated SLA for staleness is the `MAX_DATA_STALENESS`.

Which is a constant set at 30 hours, rather than the purported 24 hours. Thus, a `RedStone` feed may technically be updated past the expected 1441 minutes threshold and still be considered a valid update by the `RedStone` contracts.

Recommendation

Be aware of this 30 hour staleness validation on the `RedStone` side, and consider having a per-feed staleness threshold as a part of the remediation to H-01.

Resolution

Ethena Team: Resolved.

L-15 | Peg Arb Affected By Fees

Category	Severity	Location	Status
Unexpected Behavior	Low	Global	Acknowledged

Description

The xUSD protocol hopes to maintain the `pegPrice` while maintaining overcollateralization based on the fees charged on mint/redeem.

One of the main mechanisms for maintaining the peg is a mint/redeem arbitrage loop which would ideally keep the spot price of xUSD tied to the configured `pegPrice`.

For example, if the spot price of xUSD were currently $0.98 on a Uniswap pool then it could be bought from Uniswap at $0.98 and redeemed for $1 of collateral for a risk free profit. This trade would be executed until there is no marginal profit on the spot AMM and the spot price is $1.

However, due to the fees applied on minting and redeeming, the arbitrage opportunity is stifled.

Consider a 1% fee, if the price of xUSD on the spot market is $0.98, then arbitrageurs will be able to buy up xUSD at $0.98 and redeem it for $0.99 worth of collateral due to a peg price of $1 and a fee of $0.01.

However, when the spot price reaches $0.99 then there is no marginal profit to gain by executing the arbitrage loop, and thus arbitrageurs will not execute the trade.

As a result there will be limited demand for xUSD on the spot market above (`pegPrice - redeemFee`), which may allow the token to actively trade at less than the peg price.

Recommendation

Be aware of these dynamics, and consider if this is acceptable for the purpose of the protocol.

Resolution

Ethena Team: Acknowledged.

L-16 | Fee Is Applied Inconsistently

Category	Severity	Location	Status
Unexpected Behavior	Low	OnChainMinting.sol: 1090-1129	Acknowledged

Description

The _getQuote() function applies a fee during mints and redeems. A more conservative approach is taken where amountOut is the lesser value between oneToOneAmountOut and oracleAmountOut. The main difference between the two is oneToOneAmountOut prices the collateral asset at exactly $1, while oracleAmountOut uses the actual price returned by the oracle. The fee is applied only to the oneToOneAmountOut calculation. So the real fee paid is max(oracleAmountOut - oneToOneAmountOut, 0).

The idea is that for mints, if the price is <$1, the user already loses some value and because of that, a smaller fee is charged, or even 0 if the depeg difference exceeds the fee. However, after the mint, the user will receive the same USD value as they have put into the protocol, besides the fees. This leads to the protocol granting free mints.

For example:

• Fee = 1%, USDC = $0.99
• User A mints for 1000 USDC ($990)
• oneToOneAmountOut = oracleAmountOut = 990xUSD
• User A received 990xUSD ($990), which is the same amount they used for minting = no fee applied

Notice that if the price was $1.1 instead:

• User A mints for 1000 USDC ($1100)
• oneToOneAmountOut = 990xUSD ($990)
• oracleAmountOut = 1100xUSD ($1100)
• fee paid = oracleAmountOut - oneToOneAmountOut = $110

So the user ended up paying both the fee and the depeg difference.

The same behavior is observed in the redeem flow. This even allows entering and exiting the system without paying any fees if one of the assets is < $1 and the other is > $1.

• Mint and redeem fees = 1%; USDC = $0.99; USDT =$ 1.01
• User A mints 1000 USDC for 990xUSD
• User A redeems 990xUSD for 980 USDT
• 980 USDT = 989.8 $USD

In result the user paid almost 0 fees.

Recommendation

Consider using the netAmountIn which accounts for deducted fees in the oracleAmountOut calculation. This way even when a stablecoin depegs the overcollateralization ratio by the fee percentage is maintained upon mint/redeem actions.

Furthermore, when the stablecoin going in as collateral is depegged upwards during a mint or when the stablecoin coming out as redeemed assets is depegged downwards during a redeem, the user is hurt additionally by the fees above and beyond the desired collateralization ratio of the protocol. This may be acceptable to conservatively price the stablecoins received, however it should be considered whether this is intended or not.

Resolution

Ethena Team: Acknowledged.

L-17 | getBenefactorFeesForCollateral Skips Defaults

Category	Severity	Location	Status
Unexpected Behavior	Low	OnChainMinting.sol: 1170-1178	Resolved

Description

`OnChainMinting.getBenefactorFeesForCollateral()` returns the `mintFeeByCollateral` and `redeemFeeByCollateral` for a given pair of user and collateral or 0 if the user is exempted from fees for that collateral.

When the user is not exempt and their fee is 0, `_getQuote()` will use the:

`_collateralConfig.defaultMintFee` or `_collateralConfig.defaultRedeemFee`.

Because `getBenefactorFeesForCollateral()` doesn't use these defaults, there will be a discrepancy between the returned value from the function and the actual paid fee.

Recommendation

Consider if the default fees should be included in the `getBenefactorFeesForCollateral` function.

Resolution

Ethena Team: Resolved.

I-01 | Oracle Manipulation With Inappropriate Staleness

Category	Severity	Location	Status
Gaming	Info	AggregateOracleFeed.sol	Resolved

Description

The AggregateOracleFeed contract performs a maxStaleness validation against all of the configured oracles. Some of these oracles are pull based (PythOracleFeed) while others are push based (ChainlinkOracleFeed).

The staleness validation cannot be the same across these two types of oracles because push based aggregator feeds are updated on a less frequent basis, operating on strict SLAs for price deviation thresholds that are maintained by the oracle provider, while pull based oracles have no deviation SLA and are as fresh as the prices users choose to submit to them upon usage.

It is worth noting that some Pyth feeds are considered “sponsored feeds” and are updated regularly within a deviation SLA, however these feeds are not widely supported and we are considering non-sponsored Pyth feeds in this finding. Notice that there is no USDT sponsored feed on Ethereum for example.

A Chainlink aggregator feed that has a heartbeat window of 4 hours may have a price update that is 3 hours old but still perfectly acceptable given that the oracle SLA is e.g. 0.50% of deviation. For that reason, the maximum staleness of a push based oracle must be a longer time duration which typically matches the heartbeat duration.

However, applying that same maximum staleness to a pull based oracle such as the PythOracleFeed allows users to use prices that are potentially extremely different than the current market price because there is no deviation threshold SLA with these oracles.

Consider the following scenario:
• Market price at hour 100 is $1.00
• Current market price at hour 103 is $1.02
• The heartbeat of push based Oracle A is 4 hours, with a deviation threshold of 50 BIPs
• The maxStaleness of the aggregate oracle is assigned as 4 hours, as this is the heartbeat of the push based oracle
• Push based Oracle A was updated at the 102.5 hour mark given that the market price has moved more than 50 BIPs recently
• Pull based Oracle B has not been updated since hour 98 simply due to lacking usage
• A user may provide an update to Oracle B with the price data from hour 100 and this will be considered non-stale since it is within the aggregate oracle threshold of 4 hours
• The average price is ($1.02 (current from the push based oracle A) + $1.00 (stale price from the pull based oracle B)) / 2 = $1.01
• Thus the actor has effectively manipulated the oracle system to use an inaccurate price and can then leverage this to arbitrage the OnChainMinting system.

Recommendation

Both of the PythOracleFeed and ChainlinkOracleFeed oracle contracts do have their individual staleness checks, however currently these are both hardcoded at 1 day and 1 minute.

Enforce strict staleness validations on a per-oracle level instead of general staleness validations at the aggregate oracle level, and require that pull based feeds such as the PythOracleFeed have a very short staleness tolerance of at the very maximum a few minutes to ensure up to date pricing.

Notice that this will require users to provide a Pyth update using the hermes API, which is the expected interaction flow for the non-sponsored feeds.

Resolution

Ethena Team: Resolved.

I-02 | Ethena Minter Used To Exit Depegged Collateral

Category	Severity	Location	Status
Warning	Info	OnChainMinting.sol	Acknowledged

Description PoC

In the event that one collateral asset in the backing basket of assets depegs, the Ethena Minter contract may be used to exit this collateral asset to the detriment of other depositors.

Consider the following scenario:
• Assume fees are excluded for the two actors
• The minter is backed by two collaterals, USDT and USDC.
• USDC depegs to 90 cents
• `pegPrice` for the synthetic stablecoin is 1e18
• Alice deposits 100 USDT and receives 100e18 synthetic stablecoins
• Bob deposits 100 USDC and receives 90 synthetic stablecoins
• Bob redeems 90 synthetic stablecoins for 90 USDT
• Alice redeems 10 synthetic stablecoins for 10 USDT
• Alice redeems 90 synthetic stablecoins for 90 USDC
• Alice received (90 * $0.90) + (10 *$ 1) = $91 from her redemption, even though she had originally deposited USDT
• Bob does not benefit in this scenario
• 9 USDC is left in the system with no corresponding synthetic stablecoins to redeem it.

In this scenario, where a user deposits a stablecoin that has already depegged, there is no gain for that user due to the defensive selection of the worse exchange rate for the user in the `_getQuote` function.

However in scenarios where many users have already deposited a variety of collaterals at the time when one collateral depegs, there may be a rush to withdraw the non-depegged collateral assets to avoid being negative impacted.

Recommendation

This risk of depeg is known to the Ethena team. It may be accepted that a depeg scenario will require manual intervention, however be aware of the current behavior where there may be a "bank run" and the final redeemers will be affected the most rather than the depeg affecting all xUSD holders at a fraction of the impact.

Such an underlying collateral depeg may result in a depeg of xUSD itself given that the arbitrage loop breaks when only depegged collateral assets remain.

Resolution

Ethena Team: Acknowledged.

I-03 | Unnecessary Zero Configuration Handling

Category	Severity	Location	Status
Gas Optimization	Info	OnChainMinting.sol	Resolved

Description

In the `_validateBenefactorEpochLimits` function the validation is only performed if the provided `max` value is greater than zero.

This is likely because the benefactor's individual epoch limit configuration is optional. However in the case where the benefactor's individual epoch limit configuration is unconfigured and left as zero, the `defaultBenefactorMaxMintPerEpoch` is used in its place.

Therefore, unless the `defaultBenefactorMaxMintPerEpoch` is configured as zero, the `_validateBenefactorEpochLimits` function can never receive a `max` value of zero.

And there already exist validations that ensure that the `defaultBenefactorMaxMintPerEpoch` cannot be configured to zero.

The same applies to the `_validateBenefactorPeriodLimits` function.

Recommendation

Consider removing the zero case handling from the `_validateBenefactorEpochLimits` and `_validateBenefactorPeriodLimits` functions.

Resolution

Ethena Team: Resolved.

I-04 | Duplicated Functions

Category	Severity	Location	Status
Unexpected Behavior	Info	OnChainMinting.sol	Resolved

Description

The `getCurrentEpochEnd` and `getEpochEndTimestamp` functions as well as the `getCurrentPeriodEnd` and `getPeriodEndTimestamp` functions, have the same implementation.

Recommendation

Consider deduplicating these functions.

Resolution

Ethena Team: Resolved.

I-05 | defaultBenefactorMaxRedeemPerPeriod Typo

Category	Severity	Location	Status
Typo	Info	OnChainMinting.sol	Resolved

Description

In the documentation for the `defaultBenefactorMaxRedeemPerPeriod` and `defaultBenefactorMaxRedeemPerEpoch` functions, the `NatSpec` provides that these functions return the `defaultMaxRedeemPerPeriod` and `defaultMaxRedeemPerEpoch` respectively.

However this is not the same format that is used to describe the return value for the `defaultBenefactorMaxMintPerPeriod` and `defaultBenefactorMaxMintPerEpoch` functions.

Recommendation

Instead to follow the existing format, the return value should be stated as `defaultBenefactorMaxRedeemPerPeriod` and `defaultBenefactorMaxRedeemPerEpoch` in the `NatSpec`.

Resolution

Ethena Team: Resolved.

I-06 | DEFAULT_ADMIN_ROLE Restriction Bypass

Category	Severity	Location	Status
Access Control	Info	OnChainMinting.sol	Acknowledged

Description

The `enableBenefactor()` and `enableCollateral()` functions in `OnChainMinting` set the `isActive` flag of a benefactor or collateral to `true`. The functions can only be called by addresses with the `DEFAULT_ADMIN_ROLE`.

However, the `addBenefactor()` and `addCollateral()` functions callable by the `BENEFACTOR_MANAGER_ROLE` and `COLLATERAL_MANAGER_ROLE` can be used to activate the benefactor or the collateral, bypassing the `DEFAULT_ADMIN_ROLE` restriction.

Recommendation

Consider checking if a benefactor or collateral have already been added and revert if they have.

Resolution

Ethena Team: Acknowledged.

I-07 | OFT's Minters() Function Returns Minter Status

Category	Severity	Location	Status
Best Practices	Info	xUSDOFTUpgradeable.sol: 113-116, 124-127, 134-137	Resolved

Description

The `xUSDOFTUpgradeable.minters(address)` function's name implies the returned value will be the minters of the token, however the function returns whether the given account is a minter or not.

The same is true for `blacklist()` and `blacklister()`

Recommendation

Consider renaming the functions to `isMinter()`, `isBlacklisted()`, `isBlacklister()`.

Resolution

Ethena Team: Resolved.

I-08 | Inconsistency Between Tokens Mint Functions

Category	Severity	Location	Status
Best Practices	Info	xUSD.sol: 158-163 xUSDOFTUpgradeable.sol: 286	Resolved

Description

The `xUSDOFTUpgradeable.mint()` function reverts if the minted amount is 0.

```
if (amount == 0) revert OFTErrors.InvalidAmount();
```

On the other hand, `xUSD.mint()` doesn't perform such a validation and instead calls the internal `_mint()` with the given amount.

```
function mint(address to, uint256 amount) external {
xUSDStorage storage $ = _getxUSDStorage();
if (!$.minters[msg.sender]) revert OnlyMinter();
_mint(to, amount);
emit Minted(to, amount, msg.sender);
}
```

Recommendation

Consider performing the check in `xUSD` as well

Resolution

Ethena Team: Resolved.

I-09 | Burned Events Emitted Only For burnFrom()

Category	Severity	Location	Status
Events	Info	xUSD.sol; xUSDOFTUpgradeable.sol	Resolved

Description

`OFTEvents.Burned()` and `IxUSD.Burned()` are emitted only when the `burnFrom()` function is called, but users can also use `burn()`.

Recommendation

Double check if the events should be emitted in the `burn()` functions. If yes, add it. If no, consider changing the `NatSpec` because it currently says `Emitted when tokens are burned`.

Resolution

Ethena Team: Resolved.

I-10 | 0xdead Shouldn't Be Blacklisted

Category	Severity	Location	Status
Informational	Info	xUSDOFTUpgradeable.sol	Acknowledged

Description

If address(0xdead) is blacklisted in any of the xUSDOFTUpgradeable contracts, all crosschain transfers to address(0) will be failing. That's because OFTUpgradeable._credit() redirects address(0) transfers to address(0xdead).

```
if (_to == address(0x0)) _to = address(0xdead); // _mint(...) does not support address(0x0)
_mint(_to, _amountLD);
```

Then _mint() will invoke _update() and if 0xdead is blacklisted, the transaction will revert.

Recommendation

Keep that in mind when modifying the blacklist

Resolution

Ethena Team: Acknowledged.

I-11 | AggregateOracleFeed Always Rounds Down

Category	Severity	Location	Status
Best Practices	Info	AggregateOracleFeed.sol: 255	Acknowledged

Description

In the `getPrice` function the `averagePrice` result always uses round down division no matter what the resulting oracle price is to be used for.

When depositing collateral, it is more advantageous to the user that the oracle price reports a higher price, so in this case rounding down is appropriate.

However when withdrawing collateral, it is more advantageous to the user that the oracle prices the output collateral lower, so in this case rounding up would be appropriate.

Furthermore, within the oracle contracts that make up the `AggregateOracleFeed`, the `PythOracleFeed` and the `ChainlinkOracleFeed`, round down division is used to convert the decimals of the resulting price to 18.

In these cases the same rounding logic ought to apply to round against the user and in the favor of the protocol whenever possible.

Recommendation

Consider adding a boolean parameter to indicate which direction the `AggregateOracleFeed` and the subfeeds within it ought to round.

Resolution

Ethena Team: Acknowledged.

I-12 | USDT Has A Fee On Transfer Feature

Category	Severity	Location	Status
Unexpected Behavior	Info	OnChainMinter.sol	Acknowledged

Description

`USDT`, one of the supported collateral tokens by the minter, has a fee on transfer feature that is currently disabled.

If it's ever enabled, the accounting of the minter will behave incorrectly because the contract assumes the whole amount is being transferred to it.

Recommendation

Consider making the contract work correctly with fee on transfer tokens.

Resolution

Ethena Team: Acknowledged.

I-13 | Lack Of Admin Validation In PythOracleFeed

Category	Severity	Location	Status
Validation	Info	PythOracleFeed.sol: 113	Resolved

Description

The constructor in `PythOracleFeed.sol` does not validate `_admin != address(0)` before granting `DEFAULT_ADMIN_ROLE`.

If deployed with a zero admin, admin-only functions like `setMaxConfidence` become permanently inaccessible, preventing parameter updates and incident response.

Recommendation

Add a zero-address check for `_admin` in the constructor and revert if provided as `address(0)`.

Resolution

Ethena Team: Resolved.

I-14 | A Single Oracle Can Cause OOG Revert

Category	Severity	Location	Status
Informational	Info	AggregateOracleFeed.sol	Acknowledged

Description

The `AggregatorOracleFeed` loops through each oracle and executes a call to `getPrice()` wrapped in a `try/catch`.

An oracle can execute all of the 63/64th of the gas forwarded and leave the aggregator with insufficient amount to complete the rest of the call.

Furthermore, it can revert with a long data that has to be copied to memory in the `catch` block, again causing an OOG.

Recommendation

Make sure to add only `PythOracleFeed` and `ChainlinkOracleFeed` as oracles.

Resolution

Ethena Team: Acknowledged.

I-15 | OracleStale Emitted For Prices In Future

Category	Severity	Location	Status
Best Practices	Info	AggregateOracleFeed.sol: 223	Resolved

Description

When an oracle returns a timestamp more than `MAX_FUTURE_TIMESTAMP_TOLERANCE` seconds in the future, the code emits `OracleStale`.

This is semantically incorrect and can mislead monitoring/alerting systems diagnosing issues.

Recommendation

Consider emitting a more accurate event

Resolution

Ethena Team: Resolved.

I-16 | Inaccurate NatSpec For OFT Events And Errors

Category	Severity	Location	Status
Best Practices	Info	OFTEvents.sol: 58-60,68	Resolved

Description

The `OFTEvents.BlacklistRedirect()` event docs state the `redirectedTo` is the owner/treasury, while the implementation redirects to the `rescueRecipient`.

This inconsistency can mislead operators and off-chain indexers, complicating incident response and audits.

The `NatSpec` for `OFTErrors.BlacklistException()` says it is thrown when setting rate limits by a non-owner, but the error is actually used to signal blacklist violations in `_update()`.

Recommendation

Update the event `NatSpecs` to reflect that redirection targets `rescueRecipient` and `BlackListException()` is thrown when a blacklisted entity participates in a transfer.

Resolution

Ethena Team: Resolved.

I-17 | Benefactor Mint Fee Cannot Be Changed

Category	Severity	Location	Status
Validation	Info	OnChainMinting.sol: 636	Resolved

Description

OnChainMinting.setBenefactorMintFee() reverts if the collateral is not active.

```
if (!collateralState[collateral].config.isActive) {
revert CollateralNotSupported(collateral);
}
```

This makes it impossible to change the mint fee for a disabled collateral. In contrast, the check is not present in setBenefactorRedeemFee().

Recommendation

Consider removing the check from setBenefactorMintFee()

Resolution

Ethena Team: Resolved.

I-18 | Event Parameters Documentation Mismatch

Category	Severity	Location	Status
Best Practices	Info	IAggregateOracleFeed.sol: 123-124	Resolved

Description

The event `IAggregateOracleFeed.OracleInvalidPrice()` is documented with parameters in the order `(bound, price)`, but the event signature is `(price, bound)`, and emissions follow the signature order.

Off-chain consumers relying on the `NatSpec` docs (or positional assumptions) can invert values and misdiagnose whether the violation was lower or upper bound, hindering incident response.

Recommendation

Align documentation with the actual event signature.

Resolution

Ethena Team: Resolved.

I-19 | Fee Features Unsupported

Category	Severity	Location	Status
Documentation	Info	OnChainMinting.sol	Resolved

Description

In the documentation several fee features such as `Volume Discounts` and `Market Conditions` based fees are mentioned but not supported in the code.

Recommendation

Consider if these fee features are intended to be supported for this release or confirm if these are future features.

Resolution

Ethena Team: Resolved.

I-20 | Lacking SafeCast Usage

Category	Severity	Location	Status
Best Practices	Info	OnChainMinting.sol	Resolved

Description

In the `_getQuote` function the resulting `oracleAmountOut` and `oneToOneAmountOut` values are downcasted to uint128 without using `SafeCast`.

Although these values should never be larger than the maximum `uint128` it is a best practice to revert if this case ever would arise out of an abundance of caution.

Recommendation

Consider using `SafeCast` for these instances.

Resolution

Ethena Team: Resolved.

I-21 | addCollateral Can Be Used Inappropriately

Category	Severity	Location	Status
Unexpected Behavior	Info	OnChainMinting.sol: 482	Resolved

Description

In the `addCollateral` function the check if the collateral already exists is based upon whether the `isActive` value is true.

However, the `isActive` value can be set to false for a collateral that is de-activated and is intended to be activated again in the future. In this case the collateral can be errantly “re-added” with the `addCollateral` function, since it sees this collateral as having never been configured.

The `updateCollateral` function should be used instead in these cases to edit collateral configurations after they have already been added.

Recommendation

Instead of relying on isActive, the `addCollateral` function should validate that the `collateralCfg.custodianAddress` value is equal to zero for the collateral configuration to ensure that only a collateral that truly has not been configured is being added.

Resolution

Ethena Team: Resolved.

I-22 | removeCollateral Unexpectedly No-ops

Category	Severity	Location	Status
Unexpected Behavior	Info	OnChainMinting.sol: 503	Acknowledged

Description

The `removeCollateral` function silently completes execution in the event that the collateral is errantly not configured.

This is different from the behavior of the other collateral configuration related functions which instead revert in the case that the collateral is in an unexpected state.

Recommendation

Consider if the `removeCollateral` function should behave similarly to the other collateral configuration functions and revert as well.

Resolution

Ethena Team: Acknowledged.

I-23 | Missing Benefactor Address Validation

Category	Severity	Location	Status
Validation	Info	OnChainMinting.sol: 567	Acknowledged

Description

The `removeBenefactor` function lacks an `onlyValidAddress(benefactor)` modifier unlike the other benefactor configuration functions.

Recommendation

Consider adding an `onlyValidAddress(benefactor)` modifier to the `removeBenefactor` function.

Resolution

Ethena Team: Acknowledged.

I-24 | Opaque BenefactorConfigUpdated Event

Category	Severity	Location	Status
Events	Info	OnChainMinting.sol	Acknowledged

Description

The `BenefactorConfigUpdated` event simply emits the address of the benefactor and provides no further information about the configuration that was made.

This is unlike other configuration events such as the `CollateralConfigUpdated` event which emits the `CollateralConfig` object that was used to update the configuration.

Recommendation

Consider including information in the `BenefactorConfigUpdated` event about the configuration that was made or the final state of the benefactor config.

Resolution

Ethena Team: Acknowledged.

I-25 | Missing Signer Address Validation

Category	Severity	Location	Status
Validation	Info	OnChainMinting.sol	Acknowledged

Description

In the `removeDelegatedSigner` function there is no `onlyValidAddress(signer)` modifier as there is with the `setDelegatedSigner` function.

Recommendation

Consider adding the `onlyValidAddress(signer)` modifier to the `removeDelegatedSigner` function.

Resolution

Ethena Team: Acknowledged.

I-26 | rescueRecipient Can Be Blacklisted

Category	Severity	Location	Status
Informational	Info	xUSDOFTUpgradeable.sol: 384-414	Resolved

Description

The xUSDOFTUpgradeable contract implements a blacklist redirection mechanism in the _credit() function to prevent LayerZero double-mint vulnerabilities.

When a cross-chain transfer targets a blacklisted recipient, tokens are redirected to the rescueRecipient address instead of reverting.

However, if the rescueRecipient itself becomes blacklisted, all cross-chain transfers to the blacklisted addresses will fail until either the recipient is removed from the blacklist or a new rescueRecipient is set.

Recommendation

Consider adding validation in the setRescueRecipient function as well as the addToBlacklist function that prevents the rescueRecipient from becoming a blacklisted address.

Resolution

Ethena Team: Resolved.

I-27 | Total Minted/redeemed Per State Going Over Limit

Category	Severity	Location	Status
Informational	Info	OnChainMinting.sol	Acknowledged

Description

The global, collateral and benefactor states for epochs and periods have limits which can be changed by actors with the appropriate roles. If a lowering of the limit happens, it's possible to end up with a state where `mintedIn*` or `redeemedIn*` are over the new limit.

This state is not dangerous since further mints/redeems would just fail, but it may be unexpected for third-party integrators.

Recommendation

Document the possibility of this peculiar state.

Resolution

Ethena Team: Acknowledged.

I-28 | Limit Reverts May Be Duplicated

Category	Severity	Location	Status
Best Practices	Info	OnChainMinting.sol	Resolved

Description

Whenever an epoch or period limit is violated, the transaction reverts with an appropriate error. For example, if the mint limit for a global period is exceeded, the following error is thrown:

```
revert GlobalMaxMintPerPeriodExceeded(assetAmount, currentTotal, max, currentPeriod)
```

All of the errors of this type accept either `currentEpoch` or `currentPeriod`, but no duration. It's possible to have the same epoch or period id for different durations, therefore some of the errors may be duplicated.

Recommendation

Consider adding the durations to the revert data as well.

Resolution

Ethena Team: Resolved.

I-29 | Fee Collection Rounds Down

Category	Severity	Location	Status
Rounding	Info	OnChainMinting.sol: 1089	Resolved

Description

In the `_getQuote` function the `feeAmount` calculation uses round down division instead of round up division.

Especially because the fee affects the collateralization of xUSD, it would be best to round in favor of the protocol and against the user.

Recommendation

Consider using roundup division for the `feeAmount` computation

Resolution

Ethena Team: Resolved.

I-30 | Benefactor Addresses Must Handle Tokens

Category	Severity	Location	Status
Validation	Info	OnChainMinting.sol	Resolved

Description

In the `_validateBenefactor` function for all benefactor configurations the benefactor themselves are always allowed as an `order.beneficiary` assignment.

This means that the benefactor should always be able to handle any of the collateral tokens that could possibly be redeemed or xUSD itself, even just to handle the case where an order accidentally assigned the benefactor as the beneficiary.

Recommendation

Consider removing the default whitelisting of the benefactor themselves from the allowed beneficiary in the `_validateBenefactor` function and requiring that if the benefactor wishes to receive tokens directly that they must explicitly opt-in to this.

Otherwise, clearly document that this is assumed to be supported at the `OnChainMinting` contract level.

Resolution

Ethena Team: Resolved.

I-31 | GetQuote() Does Not Validate Price Freshness

Category	Severity	Location	Status
Math	Info	OnChainMinting.sol	Resolved

Description

The function `getQuote()` calls the internal function `_getQuote()`, which fetches the Oracle feed as well as an `updatedAt` timestamp.

The view function `getQuote()`, which is intended to fetch quotes without executing orders, does not validate the `updatedAt`.

This means that users could potentially get outdated prices, leading to a potential difference in price when executing orders.

Recommendation

Consider if this is intended. If not, then consider informing the users of the potential discrepancy in user-facing documentation.

Resolution

Ethena Team: Resolved.

I-32 | Collateral Decimals Can Be Updated

Category	Severity	Location	Status
Validation	Info	OnChainMinting.sol	Resolved

Description

The `updateCollateralConfig` function accepts a `CollateralConfig` object which will become the new configuration for the collateral.

The `CollateralConfig` object includes a decimals value which assigns the decimals which are used for the underlying collateral.

This decimals value however should not be changed from the original assignment as collateral tokens will never change their decimals denomination.

Recommendation

Consider restricting the decimals from being updated to a different value in the `updateCollateralConfig` function.

Additionally, guardrails can be added in the `addCollateral` function to validate that the returned `decimals()` value from the collateral token matches the decimals being configured, noting that this restricts the system to only be compatible with collateral tokens that implement the `decimals` external function.

Resolution

Ethena Team: Resolved.

I-33 | getQuote Can Be Marked External

Category	Severity	Location	Status
Best Practices	Info	OnChainMinting.sol	Resolved

Description

The `getQuote` function is marked as `public` but is not referenced within the `OnChainMinting` contract and can therefore be marked as `external`.

Recommendation

Mark the `getQuote` function as `external`.

Resolution

Ethena Team: Resolved.

I-34 | Unnecessary Optimizations

Category	Severity	Location	Status
Informational	Info	Global	Acknowledged

Description

Since Solidity 0.8.22, the compiler performs advanced range analysis to determine when overflow checks can safely be omitted.

In a typical loop increment scenario such as for (`uint256 i = 0; i < length; i++`), the compiler can prove that i will not overflow a `uint256` and therefore, it automatically removes the checks.

Manually placing increments in an unchecked block is now redundant, making the code less clear without providing a meaningful gas optimization.

Recommendation

Throughout the codebase, replace the unnecessary unchecked { ++i; } instances with a simple ++i in the for-loop declaration, allowing the compiler's range analysis to handle overflow optimizations automatically.

This cleaner style increases code clarity while maintaining code efficiency in Solidity 0.8.22 and above.

Resolution

Ethena Team: Acknowledged.

I-35 | End Timestamp Functions Return The Next Start

Category	Severity	Location	Status
Informational	Info	OnChainMinting.sol: 1405-1418	Acknowledged

Description

`OnChainMinting.getEpochEndTimestamp()` and `OnChainMinting.getPeriodEndTimestamp()` aim to return the timestamp when the current epoch/period ends. The calculations are as follows:

```
uint256 currentEpoch = _getCurrentEpoch(globalState);
return (currentEpoch + 1) * globalState.config.epochDuration;
```

The resulting timestamp will be the start of the next epoch/period, which may be unexpected for third-party integrators.

Recommendation

Consider either documenting this behavior or subtracting 1 from the end result.

Resolution

Ethena Team: Acknowledged.

I-36 | COLLATERAL_MANAGER_ROLE Can Change The Oracle

Category	Severity	Location	Status
Trust Assumptions	Info	OnChainMinting.sol: 535-536	Acknowledged

Description

`OnChainMinter.updateCollateralConfig()` can be invoked by addresses with the `COLLATERAL_MANAGER_ROLE` to update the configuration for a given collateral.

One of the fields in the `CollateralConfig` struct is the `oracleFeed` used to retrieve the price when minting and redeeming.

The ability to change the oracle at any time creates risks for the protocol, since it can be set at as any address and even return fake data.

Recommendation

Consider whether the `oracleFeed` should be modifiable. If yes, you can give that right to the `DEFAULT_ADMIN_ROLE` and enforce that `oracleFeed` is not changed in `updateCollateralConfig()`.

Resolution

Ethena Team: Acknowledged.

I-37 | Beneficiary Can Be Set For Inactive Benefactor

Category	Severity	Location	Status
Informational	Info	OnChainMinting.sol: 809-820	Acknowledged

Description

`OnChainMinting.setApprovedBeneficiary()` is a permissionless function that allows benefactors to add or remove beneficiaries.

The function doesn't check if the benefactor is active, which means a benefactor that has never been approved can set their beneficiary and emit the corresponding events - `BeneficiaryApproved` and `BeneficiaryRemoved`.

This is not aligned with other functions which modify the beneficiary state, for example `setDelegatedSigner()`, and it can cause confusion for any third-party integrators if they expect invalid benefactors to have empty config.

Recommendation

Consider validating the `isActive` flag when adding a new beneficiary, but still allow removing the old one in order not to block benefactors from removing beneficiaries when they are temporarily disabled.

Resolution

Ethena Team: Acknowledged.

I-38 | Regular xUSD Does Not Implement Blacklist

Category	Severity	Location	Status
Warning	Info	xUSD.sol	Resolved

Description

In the normal xUSD implementation there is no blacklist logic.

Recommendation

Consider if blacklist logic should be implemented for the basic, non-OFT xUSD implementation.

Resolution

Ethena Team: Resolved.

I-39 | Fee Exempt Mints Reduce Collateralization

Category	Severity	Location	Status
Warning	Info	OnChainMinting.sol	Acknowledged

Description

Benefactors who are fee exempt reduce the overall collateralization of the system when they mint, because fees are considered a part of the collateral backing xUSD.

Consider the following example:

• The existing system has 110 USDC collateral and 100 xUSD supply.

• The collateralization rate is 110%.

• A fee exempt benefactor deposits 100 USDC to receive 100 xUSD.

• The system now has 210 USDC collateral and 100 xUSD.

• The collateralization rate has dropped to 105%.

Recommendation

Be aware of this collateralization reduction and consider it when allowing benefactors to be fee exempt.

Resolution

Ethena Team: Acknowledged.

I-40 | Peg Updates Are Sandwichable

Category	Severity	Location	Status
Gaming	Info	Global	Acknowledged

Description

When the peg price of the xUSD asset is updated in the `OnChainMinting` contract a risk free sandwich arbitrage opportunity is created where a user may execute a mint order right before the peg price increase and execute a redeem order right after the peg price increase, gaining the difference in the mint/redemption rate for xUSD.

Recommendation

Be sure to use a private RPC when updating the peg price. Furthermore, be aware that benefactors and signers have the ability to arbitrage this update. It may be prudent to fully disable minting and redeeming before making the update and broadcasting the peg increase.

Resolution

Ethena Team: Acknowledged.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Same Block OFT Transfers Are Blocked	DoS	Medium	Resolved
L-01	Missing Check For Maximum Oracles Allowed	Validation	Low	Resolved
L-02	Missing rescueBlacklistedTokens Function	Unexpected Behavior	Low	Resolved
I-01	Discrepancy In The Emission Of Burned Event	Events	Info	Resolved
I-02	Inconsistency Between Tokens Mint Functions	Validation	Info	Resolved
I-03	Incorrect Constant Formatting	Best Practices	Info	Resolved
I-04	Redundant Inheritance Pattern	Best Practices	Info	Resolved
I-05	isMinter Function Discrepancy	Best Practices	Info	Resolved
I-06	removeCollateral Misleading Documentation	Documentation	Info	Resolved
I-07	Inaccurate Validation Documentation	Documentation	Info	Resolved
I-08	Invalid SPDX Header	Best Practices	Info	Resolved
I-09	Missing Collateral Value	Events	Info	Resolved
I-10	Rate Limit Caps Cause Unexpected Behavior	Unexpected Behavior	Info	Resolved

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
I-11	Lacking Min/Max Oracle Price Validation	Validation	Info	Resolved
I-12	Outdated Interface	Best Practices	Info	Resolved

M-01 | Same Block OFT Transfers Are Blocked

Category	Severity	Location	Status
DoS	Medium	RateLimiterUpgradeable.sol: 138	Resolved

Description

When OFTs are being sent to another chain, `_amountCanBeSent()` is executed to enforce rate limiting.

A new check was added to avoid overflow when `limit` and `timeSinceLastDeposit` are multiplied. A division by `timeSinceLastDeposit` is performed, but it's not checked whether its value is 0 or not.

```
uint256 timeSinceLastDeposit = block.timestamp - _lastUpdated;
if (_limit > type(uint256).max / timeSinceLastDeposit) {...}
```

This means only 1 `send()` operation can be executed per block, as the rest of them will revert. This behavior can be weaponized by sending a small amount of tokens to DOS the `send` feature for the whole block.

Recommendation

Handle the case where `timeSinceLastDeposit == 0` separately.

Resolution

Ethena Team: Resolved.

L-01 | Missing Check For Maximum Oracles Allowed

Category	Severity	Location	Status
Validation	Low	AggregateOracleFeed.sol	Resolved

Description

A new constant `MAX_NUMBER_OF_ORACLES = 4` was added to `AggregateOracleFeed` to limit the number of oracles being used. This is needed because the `Math` library supports at most 4 oracles.

While `addOracle()` reverts if the array exceeds 4 oracles, there is no such check in the constructor. Because of this an aggregator can be deployed with more than 4 oracles and result in reverts in all cases.

Recommendation

Validate the number of oracles being used in the constructor.

Resolution

Ethena Team: Resolved.

L-02 | Missing rescueBlacklistedTokens Function

Category	Severity	Location	Status
Unexpected Behavior	Low	xUSD.sol	Resolved

Description

In the `xUSDOFTUpgradeable` contract there is a `rescueBlacklistedTokens` function which rescues funds from a blacklisted address.

Blacklist functionality has been introduced to the xUSD contract, however such a rescue function has not been added to `xUSD`.

Recommendation

Consider adding a similar `rescueBlacklistedTokens` function to the xUSD contract.

Resolution

Ethena Team: Resolved.

I-01 | Discrepancy In The Emission Of Burned Event

Category	Severity	Location	Status
Events	Info	xUSD.sol: 270	Resolved

Description

`xUSDOFTUpgradeable.burnFrom()` emits the `Burned()` event only if the burner is a minter. In contrast, in `xUSD` the event will be emitted for all burns, no matter the caller.

Recommendation

Consider emitting `Burned()` only on burns from a minter as well.

Resolution

Ethena Team: Resolved.

I-02 | Inconsistency Between Tokens Mint Functions

Category	Severity	Location	Status
Validation	Info	xUSD.sol	Resolved

Description

As described in I-08 of the main review, there is a discrepancy between the mint() functions in xUSD and xUSDOFTUpgradeable - the former allows 0 amount mints, while the latter reverts.

Furthermore, the xUSDOFTUpgradeable implementation does not enforce the blacklist like the xUSD mint function does.

The remediation commit provided, 14e5dbacf9aa235c165d39cb068afc8cf5cfd27c, doesn't fix the issue.

Recommendation

Revert in xUSD.mint() if the amount is 0.

Resolution

Ethena Team: Resolved.

I-03 | Incorrect Constant Formatting

Category	Severity	Location	Status
Best Practices	Info	Global	Resolved

Description

Throughout the onchain minting project constants are correctly formatted in screaming snake case. However, the storage location variable declarations do not follow this formatting.

Recommendation

Consider updating the formatting of the following variables to reflect the screaming snake case formatting for constants:

• `xUSDOFTUpgradeableStorageLocation`

• `OFTOwnable2StepUpgradeableStorageLocation`

• `RateLimiterUpgradeableStorageLocation`

• `xUSDStorageLocation`

Resolution

Ethena Team: Resolved.

I-04 | Redundant Inheritance Pattern

Category	Severity	Location	Status
Best Practices	Info	xUSD.sol	Resolved

Description

The xUSD contract includes several redundant inheritances of Initializable and `ERC20Upgradeable` which are not strictly necessary and become inherited through the `UUPSUpgradeable` and `ERC20BurnableUpgradeable` contracts respectively.

The `xUSDOFTUpgradeable` contract does not include such redundant inheritances.

Recommendation

Consider removing the redundant Initializable and `ERC20Upgradeable` inheritances from the xUSD contract.

Resolution

Ethena Team: Resolved.

I-05 | isMinter Function Discrepancy

Category	Severity	Location	Status
Best Practices	Info	xUSD.sol	Resolved

Description

In the `xUSD` contract the view function which exposes a predicate to check if an account is a whitelisted minter is called `minters`, however in the `xUSDOFTUpgradeable` contract this function is called `isMinter`.

Recommendation

Consider aligning the implementations by renaming the `xUSD` function to `isMinter`.

Resolution

Ethena Team: Resolved.

I-06 | removeCollateral Misleading Documentation

Category	Severity	Location	Status
Documentation	Info	OnChainMinting.sol	Resolved

Description

Finding I-22 from the main review points out that the `removeCollateral` behavior differs from other configuration functions and no-ops instead of reverting.

The Ethena team has acknowledged this as expected, however the documentation for the `removeCollateral` function says, `Reverts if collateral doesn't exist`.

However, this is not the case, as acknowledged by I-22.

Recommendation

Update the documentation of the `removeCollateral` function to reflect the desired no-op behavior.

Resolution

Ethena Team: Resolved.

I-07 | Inaccurate Validation Documentation

Category	Severity	Location	Status
Documentation	Info	OnChainMinting.sol	Resolved

Description

The documentation for the `_validateBenefactorEpochLimits` and `_validateBenefactorPeriodLimits` functions still suggests that it `Only validates if max > 0 (unlimited if max == 0)`.

However, this is no longer the case.

Recommendation

Consider updating the documentation for these functions.

Resolution

Ethena Team: Resolved.

I-08 | Invalid SPDX Header

Category	Severity	Location	Status
Best Practices	Info	AggregateOracleFeed.sol	Resolved

Description

The SPDX license identifier is set to GPL-3.30, which is not a valid SPDX expression. This can break license detection tools and compliance pipelines.

Recommendation

Change the SPDX header to a valid identifier, e.g., // SPDX-License-Identifier: GPL-3.0.

Resolution

Ethena Team: Resolved.

I-09 | Missing Collateral Value

Category	Severity	Location	Status
Events	Info	OnChainMinting.sol	Resolved

Description

In the `_getQuote` function, when `oraclePrice == 0`, `_getQuote` reverts with `InvalidOraclePrice` but passes `address(0)` as the collateral address. This makes debugging and monitoring difficult because the emitted error reports the wrong collateral.

Recommendation

Consider adding the actual collateral token address in the revert for additional details.

Resolution

Ethena Team: Resolved.

I-10 | Rate Limit Caps Cause Unexpected Behavior

Category	Severity	Location	Status
Unexpected Behavior	Info	RateLimiterUpgradeable.sol	Resolved

Description

The overflow guard caps decay to `_limit` when `_limit * timeSinceLastDeposit` would overflow `uint256`. While this avoids arithmetic errors, it can overestimate decay (treating it as a full-window decay in these cases) and thus grant more capacity than intended for small time intervals.

Although practically hard to exploit (requires extreme limit/window), if this would arise due to an extreme configuration, it may cause unexpected behavior.

Recommendation

Instead of using explicit overflow checks, consider using a math library that handles intermediate overflow such as `OpenZeppelin` `Math.mulDiv`.

Resolution

Ethena Team: Resolved.

I-11 | Lacking Min/Max Oracle Price Validation

Category	Severity	Location	Status
Validation	Info	AggregateOracleFeed.sol	Resolved

Description

In the `AggregateOracleFeed` constructor the `_minOraclePrice` and `_maxOraclePrice` is accepted and assigned to storage without checking their validity relative to each other.

Currently a `_maxOraclePrice` that is smaller than the `_minOraclePrice` is a valid configuration.

Recommendation

Consider validating that the `_maxOraclePrice` is larger than the `_minOraclePrice` in the constructor.

Resolution

Ethena Team: Resolved.

I-12 | Outdated Interface

Category	Severity	Location	Status
Best Practices	Info	IAggregateOracleFeed.sol	Resolved

Description

In the `IAggregateOracleFeed` interface the average price is still referenced instead of the now Median price being used.

Recommendation

Consider updating the documentation in the interface to avoid confusion for integrators and users.

Resolution

Ethena Team: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits